

Web API Hands-On — Answer Key

.NET Web API: REST, HTTP Verbs, Status Codes, Configuration & First API

Objectives Covered	RESTful web services and Web API vs Microservice • REST architecture features • HttpRequest/HttpResponse • HTTP action verbs • HttpStatusCode • creating a first Web API • Web API configuration files.
---------------------------	---

1. RESTful Web Service, Web API & Microservice

What is a RESTful Web Service?

A **RESTful web service** is a service that follows the principles of **REST (Representational State Transfer)** — an architectural style for designing networked applications. It exposes resources (e.g. Products, Orders) as URIs, and clients interact with those resources using standard HTTP verbs (GET, POST, PUT, DELETE).

What is Web API?

Web API is a framework (ASP.NET Web API / ASP.NET Core Web API) for building HTTP services that can be consumed by a broad range of clients — browsers, mobile apps, desktop apps, IoT devices — because it communicates over plain HTTP and can return data in JSON, XML, or other formats.

What is a Microservice?

A **microservice** is a small, independently deployable service that owns a single business capability (e.g. an Orders service, a Payments service). A Web API is often the technology used to *expose* a microservice's functionality over HTTP, but the two terms are not synonyms: Web API is about the communication mechanism, while microservice is about the architectural decomposition of an application into small, independent services.

Web Service vs Web API

Aspect	Web Service	Web API
Protocol	Works over HTTP only, typically SOAP-based	Works over HTTP and can be REST or SOAP-based
Data format	Primarily XML (SOAP envelope)	Not restricted to XML — supports JSON, XML, plain text, etc.
Hosting	Requires IIS or a similar host	Can be self-hosted (console app, Kestrel) or IIS-hosted
State	Can be stateful	Designed to be stateless (REST constraint)
Contract	Strict contract via WSDL	No formal contract required; often documented with Swagger/O

Key point ("Not restricted to send XML as response"): unlike traditional SOAP web services, a Web API action can return JSON (the modern default), XML, plain text, or even raw binary content — content negotiation lets the client choose via the Accept header.

2. Features of REST Architecture

- **Representational State Transfer (REST):** resources are identified by URIs, and their *representation* (JSON/XML) is transferred between client and server — the resource itself never leaves the server, only a

representation of its current state.

- **Stateless:** every request from the client must contain all the information the server needs to process it. The server does not store any client session state between requests — this makes REST APIs easy to scale horizontally, since any server instance can handle any request.
- **Messages:** communication happens through standard HTTP messages (requests and responses), each with a method, headers, and optionally a body — there is no proprietary messaging protocol.
- **Uniform interface:** resources are manipulated using a fixed, small set of operations (GET/POST/PUT/DELETE) and are identified by URIs, giving a predictable, self-descriptive API surface.
- **Client-Server separation:** the client (UI/consumer) and server (data/business logic) evolve independently, as long as the interface between them stays consistent.
- **Cacheable:** responses can be marked cacheable or non-cacheable, allowing clients/proxies to reuse prior responses and reduce load.

3. HttpRequest & HttpResponseMessage

HttpRequest

An **HttpRequest** represents the message a client sends to the server. It carries:

- **Method/Verb** — GET, POST, PUT, DELETE, etc., indicating the intended action.
- **URI** — the resource being addressed, e.g. /api/products/5.
- **Headers** — metadata such as Content-Type, Authorization, Accept.
- **Body** — the payload (e.g. JSON representing a new resource), present on POST/PUT requests.

HttpResponse

An **HttpResponse** is what the server sends back. It carries:

- **Status code** — a 3-digit code indicating the outcome (200, 404, 500, etc.).
- **Headers** — e.g. Content-Type describing the format of the body.
- **Body** — the resource representation (JSON/XML) or error details.

In ASP.NET Core Web API, action methods typically return an **ActionResult** (or **ActionResult<T>**), which encapsulates both the status code and the response body in a single, testable return type.

4. Types of Action Verbs (HTTP Methods)

Verb	Attribute	Purpose
GET	[HttpGet]	Retrieve a resource or a collection — safe and idempotent, no side effects.
POST	[HttpPost]	Create a new resource — not idempotent; calling it twice creates two resources.
PUT	[HttpPut]	Update/replace an existing resource entirely — idempotent.
DELETE	[HttpDelete]	Remove a resource — idempotent (deleting twice has the same end state).
PATCH	[HttpPatch]	Partially update a resource (not explicitly listed in the objectives, but commonly paired with the

Example controller declaring each verb as an attribute:

```

[Route("api/[controller]")]
[ApiController]
public class ProductsController : ControllerBase
{
    [HttpGet]
    public IActionResult GetAll() => Ok(_repo.GetAll());

    [HttpGet("{id}")]
    public IActionResult Get(int id) => Ok(_repo.Get(id));

    [HttpPost]
    public IActionResult Create([FromBody] Product product) { ... }

    [HttpPut("{id}")]
    public IActionResult Update(int id, [FromBody] Product product) { ... }

    [HttpDelete("{id}")]
    public IActionResult Delete(int id) { ... }
}

```

Each attribute maps the method to the matching HTTP verb; ASP.NET Core's routing engine uses these attributes (combined with the [Route] template) to dispatch an incoming request to the correct action.

5. Types of HttpStatusCode Used in Web API

Status Code	ActionResult Helper	Meaning
200 OK	Ok() / Ok(data)	Request succeeded; response body contains the result.
201 Created	CreatedAtAction() / Created()	A new resource was successfully created (typical POST response).
204 No Content	NoContent()	Request succeeded but there is no body to return (typical DELETE/PUT response).
400 Bad Request	BadRequest() / BadRequest(ModelState)	The request was malformed or failed validation.
401 Unauthorized	Unauthorized()	The caller is not authenticated — missing/invalid credentials.
403 Forbidden	Forbid()	The caller is authenticated but not allowed to perform this action.
404 Not Found	NotFound()	The requested resource does not exist.
500 InternalServerError	StatusCode(500, ...)	An unhandled exception occurred on the server.

Example returning different status codes via action result types:

```

[HttpGet("{id}")]
public IActionResult Get(int id)
{
    var product = _repo.Get(id);
    if (product == null)
        return NotFound(); // 404

    return Ok(product); // 200
}

[HttpPost]
public IActionResult Create([FromBody] Product product)
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState); // 400
}

```

```

_repo.Add(product);
return CreatedAtAction(nameof(Get), new { id = product.Id }, product); // 201
}

```

Returning the right status code (rather than always 200) is what makes a Web API self-descriptive and lets clients react programmatically (e.g. retry on 500, redirect to login on 401).

6. Configuration Files of Web API

ASP.NET Core (modern)

File	Purpose
Program.cs / Startup.cs	Configures the request pipeline (middleware) and registers services via dependency injection (e.g. <code>services.AddControllers()</code>).
appsettings.json	Stores application configuration such as connection strings, logging levels, and custom settings, read by <code>ConfigurationManager</code> .
appsettings.{Environment}.json	Environment-specific overrides (Development/Staging/Production), merged on top of appsettings.json.
launchSettings.json	Local development launch profiles — which port/URL to use, environment variables (e.g. ASPNETCORE_ENVIRONMENT).

Dependency Injection in Program.cs (minimal hosting model, .NET 6+):

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddDbContext<AppDbContext>(opts =>
    opts.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
builder.Services.AddScoped<IProductRepository, ProductRepository>();

var app = builder.Build();

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();

```

Explanation: `builder.Services` registers dependencies (`DbContext`, repositories, custom services) into the built-in DI container; controllers then receive them via constructor injection instead of instantiating them directly. `app.Map/Use` calls build the middleware pipeline that every request flows through, in order.

ASP.NET Web API on .NET Framework 4.5 (legacy)

File	Purpose
WebApiConfig.cs (App_Start folder)	Registers Web API routes and configuration — e.g. <code>config.Routes.MapHttpRoute(...)</code> — and is invoked by <code>Global.asax</code> .
RouteConfig.cs (App_Start folder)	Registers MVC routes for the rest of the application (non-API controllers/views); separate from Web API.
Global.asax	Application entry point; wires up <code>WebApiConfig</code> , <code>RouteConfig</code> , and other startup registrations for <code>HttpRuntime</code> .
Web.config	XML-based configuration — connection strings, app settings, module/handler registration — the base configuration for the application.

Difference in short: `WebApiConfig.cs` configures routing specifically for Web API controllers (those inheriting `ApiController`), while `RouteConfig.cs` configures routing for traditional MVC controllers — in .NET Core these are unified under a single routing system configured in `Program.cs/Startup.cs`.

7. Hands-On: First Web API Using .NET Core

Task: Create a .NET Core Web API project using the API template, generate a controller with Read/Write actions, and verify the GET action method returns the expected result.

Step 1 — Create the project

```
dotnet new webapi -n ValuesApiDemo
cd ValuesApiDemo
```

The `webapi` template scaffolds a ready-to-run ASP.NET Core Web API project with `Program.cs`, `appsettings.json`, `launchSettings.json`, and (in older template versions) a sample `WeatherForecastController` — the equivalent of the classic `ValuesController` from earlier .NET Core versions.

Step 2 — Create a controller with Read/Write actions

Add a new controller, e.g. `Controllers/ValuesController.cs`, with actions that correspond to each HTTP verb (this mirrors what the Visual Studio "Add Controller > API Controller with read/write actions" scaffolder generates):

```
using Microsoft.AspNetCore.Mvc;

namespace ValuesApiDemo.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ValuesController : ControllerBase
    {
        private static readonly List<string> _values = new() { "value1", "value2" };

        // GET api/values
        [HttpGet]
        public ActionResult<IEnumerable<string>> Get() => Ok(_values);

        // GET api/values/5
        [HttpGet("{id}")]
        public ActionResult<string> Get(int id)
        {
            if (id < 0 || id >= _values.Count) return NotFound();
            return Ok(_values[id]);
        }

        // POST api/values
        [HttpPost]
        public IActionResult Post([FromBody] string value)
        {
            _values.Add(value);
            return CreatedAtAction(nameof(Get), new { id = _values.Count - 1 }, value);
        }

        // PUT api/values/5
        [HttpPut("{id}")]
        public IActionResult Put(int id, [FromBody] string value)
        {
            if (id < 0 || id >= _values.Count) return NotFound();
            _values[id] = value;
            return NoContent();
        }

        // DELETE api/values/5
    }
}
```

```

[HttpDelete("{id}")]
public IActionResult Delete(int id)
{
    if (id < 0 || id >= _values.Count) return NotFound();
    _values.RemoveAt(id);
    return NoContent();
}
}
}

```

Structure notes: ValuesController inherits from ControllerBase (the ASP.NET Core equivalent of the legacy ApiController) — this base class provides helper methods like Ok(), NotFound(), and CreatedAtAction() used above. The [ApiController] attribute enables automatic model-state validation and binding-source inference. The [Route("api/[controller]")] attribute maps requests to /api/values, with [controller] substituted from the class name.

Step 3 — Run the application

```
dotnet run
```

The console prints the listening URL, typically something like `https://localhost:7031`. If the project template includes Swagger/OpenAPI (enabled by default in recent templates), navigating to `https://localhost:7031/swagger` opens an interactive UI listing every action — a fast way to try GET/POST/PUT/DELETE without a separate tool.

Step 4 — Verify the GET action returns the expected result

Test with a browser, curl, or Postman:

```
curl https://localhost:7031/api/values
```

Expected response — HTTP 200 OK with body:

```
["value1", "value2"]
```

```
curl https://localhost:7031/api/values/0
```

Expected response — HTTP 200 OK with body:

```
"value1"
```

If instead you request an out-of-range id (e.g. `/api/values/99`), the action returns **404 Not Found**, confirming the controller's action verbs, routing, and status-code handling all work end-to-end — the same pattern you'll extend later to talk to a real database (as in the EF Core labs) instead of the in-memory `_values` list used here.

Key Takeaways

- Web API exposes application functionality over plain HTTP, unrestricted to XML, following REST principles: stateless, resource-oriented, uniform interface.
- Action verbs (GET/POST/PUT/DELETE) map directly to controller actions via [Http*] attributes.
- IActionResult / ActionResult<T> return types let a single action communicate both a status code (200, 201, 400, 401, 404, 500, etc.) and a response body.
- Configuration is split across Program.cs/Startup.cs (pipeline + DI), appsettings.json (settings), and launchSettings.json (local dev profiles) in .NET Core — versus WebApiConfig.cs/RouteConfig.cs/Web.config in the legacy .NET Framework 4.5 model.

- The dotnet new webapi template plus a controller with [HttpGet]/[HttpPost]/[HttpPut]/[HttpDelete] actions is enough to stand up and verify a working Web API end-to-end.